

تدقيق شامل لمشروع DentaFlow

— كل الموديولات والـ Flows

مراجعة كاملة لسير العمل الفعلي في الكود (Schema + Server Actions + الصفحات) عبر ست منظومات: الدخول والصلاحيات، المرضى والملف الطبي، المواعيد والزيارات، الفواتير و workflow الموافقات، المخزون، والتقارير. الهدف: معرفة إيه اللي شغال فعلاً بأمان، وإيه اللي لسه فيه ثغرة أو ناقص.

التاريخ: 08-09-2026 طريقة المراجعة: قراءة كود مباشرة (بدون تعديل) — 5 مراجعات متوازية عدد الملاحظات: 31

31

إجمالي الملاحظات

11

ملاحظات بسيطة (Minor)

13

ملاحظات متوسطة (Moderate)

7

ثغرات حرجية (Critical)

الموديول	نسبة الاكتمال التقديرية	الحالة الأمنية
الدخول + الصلاحيات + المستخدمين + الفروع	90%	فيه ثغرة جلسة
المرضى + الملف الطبي + مخطط الأسنان	85%	غير آمن (IDOR)
المواعيد + الزيارات	88%	ثغرات معروفة قائمة
الفواتير + الموافقات	80%	Workflow قابل للالتفاف
المخزون	35%	CRUD فقط، لا حركات مخزون
التقارير	55%	أرقام حقيقية لكن ناقصة
المصروفات + مستحقات الأطباء	0%	غير مبني (Schema فقط)

01 الدخول والصلاحيات والمستخدمين والفروع

تسجيل الدخول برقم الهاتف، NextAuth v5، إدارة الأطباء/المساعدين/الفروع (SUPER_ADMIN فقط).

الملفات: `src/lib/auth.ts` · `src/proxy.ts` · `src/lib/auth-utils.ts` · `src/lib/actions/users.ts` · `src/lib/actions/branches.ts`

حرج

جلسة المستخدم المُعطل تفضل شغالة

`src/auth.ts:34,48-62` — فحص `isActive` يحصل مرة واحدة بس وقت الدخول جوه `authorize()`. الـ `session` / `jwt` مفيهاش أي إعادة استعلام من الداتابيز، فال JWT يفضل صالح لمدة صلاحيته الكاملة (شهر افتراضياً) حتى لو الأدمن عطل الحساب من `toggleUserActive` بعد كده. نفس المشكلة لو الأدمن غيّر دور (role) المستخدم — الـ `session` القديمة هتفضل شايلة الدور القديم لحد ما يعمل `logout/login` تاني.

سيناريو فعلي: موظف بيتفصل من العيادة، الأدمن بيعطل حسابه فوراً — لكن الموظف يقدر يفتح نفس المتصفح ويكمل يستخدم النظام بصلاحياته الكاملة لحد ما تنتهي صلاحية الكوكي.

متوسط

Race condition عند إنشاء/تعديل مستخدم بنفس رقم الهاتف

`src/lib/actions/users.ts:38-53, 69-93` — الفحص بيتم بـ `findUnique` منفصل قبل `create`، من غير `transaction`. ضغطتين متزامنتين بنفس الرقم بيعدّوا الفحص سوا، وبعدين يتصادموا على الـ `unique constraint` فيطلع خطأ Prisma خام (P2002) للمستخدم بدل رسالة "رقم الهاتف مستخدم بالفعل".

بسيط

الفروع المُعطلة تفضل قابلة للتعين

`src/lib/data/users.ts:14-19` — `getBranchOptions()` ما بيفلترش على `isActive: true`، فاخيار الفرع في نموذج إضافة/تعديل طبيب أو موظف بيعرض حتى الفروع المُعطلة.

بسيط

مفتاح تفعيل/تعطيل المستخدم ما بيوضحش سبب الفشل

`src/components/shared/toggle-active-switch.tsx:20-30` — عند فشل العملية (مثلاً "متقدرش تعطل حسابك" انت) الـ `switch` بيرجع مكانه بصمت من غير ما يعرض رسالة الخطأ.

regex رقم الهاتف المصري (01[0125]XXXXXXX) متطابق في كل الملفات (client + server). كل ال Server Actions الخاصة بالمستخدمين والفروع بتتحقق من `requireSuperAdmin()` فعليًا على السيرفر — مفيش أكشن بيثق في دور العميل. كلمة السر متعمول لها hash بـ bcrypt وما بترجعش أبدًا للعميل ولا بتسجل في أي log. الفروع/الأطباء اللي معندهمش علاقات بتعرض empty states سليمة بدل ما تكسر الصفحة.

02 المرضى + الملف الطبي + مخطط الأسنان

أخطر موديول في هذا التدقيق — عمليات كتابة حساسة (بيانات مريض، تاريخ مرضي، حالة أسنان) بلا فحص صلاحيات على مستوى ال Server Action.

الملفات: `src/lib/auth-utils.ts` · `dental-chart.ts` · `medical-history.ts` · `src/lib/actions/patients.ts`

حرج

تعديل مخطط الأسنان بدون أي فحص وصول

`src/lib/actions/dental-chart.ts:14-16` — `updateToothRecord` بيتأكد بس إن فيه session `if` `return (!user)`، وما بيستدعش `canStaffAccessPatient` ولا أي فحص فرع. فحص الصلاحيات على مستوى الصفحة (`getPatientById`) مالوش قيمة هنا لأن ال Server Action نفسه endpoint مباشر يُستدعى بشكل منفصل.

سيناريو فعلي: موظف في فرع المعادي يقدر يستدعي `updateToothRecord` بأي `patientId` — حتى مريض ماشافش فرعه أبدًا — ويعدّل حالة أسنانه بصمت.

حرج

تعديل التاريخ الطبي بلا أي مصادقة أو صلاحية

`src/lib/actions/medical-history.ts:14` — `upsertMedicalHistory` ما بيستدعش `getCurrentUser()` أصلًا، ولا فيه أي فحص دور أو فرع. أي مستخدم مسجّل دخول — أي دور كان — يقدر يعدّل الحساسيات والأمراض المزمنة والحمل والأدوية لأي مريض في النظام بمجرد معرفة ال ID.

حرج

إنشاء/تعديل بيانات المريض بلا مصادقة أو تحقق من الفرع

`src/lib/actions/patients.ts:24-58` — لا `createPatient` ولا `updatePatient` بيستدعوا `getCurrentUser`، و `updatePatient` كمان ما بيتحققش إن `registrationBranchId` الجديد فعلاً فرع المستخدم. فقط `deletePatient` فيه فحص دور.

سيناريو فعلي: موظف فرع أ يقدر يستدعي `updatePatient` وينقل تسجيل أي مريض لفرع تاني أو يعدّل بياناته حتى لو مريض شافه فرع ب بس.

متوسط

دالة فحص الوصول (canStaffAccessPatient) موجودة لكن مش مُستخدمة

— `src/lib/auth-utils.ts:27-39` الدالة موجودة أصلاً وبتفحص الزيارات فقط (غير متسقة مع

`patientScopeWhere` في `src/lib/data/patients.ts:13-22` اللي بتحسب كمان `registrationBranchId`

والمواعيد)، لكنها مش متصلة بأي Server Action كتابة — وده على الأرجح سبب الثغرات الثلاثة اللي فوق.

متوسط

نموذج تعديل المريض يفرض حقول لازم تبقى اختيارية (المريض العابر)

`src/lib/actions/patients.ts:8-22` و `src/components/patients/patient-form-dialog.tsx:38-52`

— ال Zod schema بتفرض `nationalId` / `gender` / `birthDate` كحقول إلزامية، رغم إن ال Prisma schema بتوضح إنها nullable خصيصاً لمريض "كشف عادي" أنشئ من الحجز مباشرة. النتيجة: مريض عابر متسجل بدون تاريخ ميلاد ما يقدرش حد يعدل بياناته الأساسية بدون ما يُجبر على استيفاء بيانات مش موجودة أصلاً.

بسيط

عدم التحقق من رقم السن (FDI)

— `src/lib/actions/dental-chart.ts:9-14,18` بيتقبل أي رقم صحيح بدون تحقق من كونه

ضمن ترقيم (11-48) FDI. رقم زي 99 أو 0 ممكن يتسجل ويطلع تصنيف فك/جانب غير منطقي أو يختفي تمامًا من شبكة ال 32 سن في الواجهة.

بسيط

حقول الحساسيات/الأمراض المزمنة بلا تنقية تكرار

— `src/components/patients/medical-history-form-dialog.tsx:57-62,96-104` إدخال نصي حر مفصول

بفواصل، بدون إزالة التكرار أو توحيد المسافات/حالة الأحرف، ف"سكر" و"سكر" ممكن يتسجلوا كعنصرين مختلفين.

سليم

نقاط قوة مؤكدة

ال upsert على `ToothRecord` (@unique) شغال صح بدون تعارض، وكل تحديث بيسجل `AuditLog` فيه
oldValue/newValue فعلياً داخل نفس ال transaction. البحث بالاسم في قائمة المرضى case-insensitive، وال
pagination شغالة. حالات المريض العابر (بدون تاريخ ميلاد/نوع) بتُعرض بأمان في صفحة الملف بفضل فحوصات
`filter(Boolean)`.

03 المواعيد + الزيارات

حجز/تعديل/إلغاء موعد، كشف التعارض، تسجيل الحضور كزيارة فعلية. هذا الموديول كان موضوع تقرير تدقيق سابق — تم التحقق من حالة كل ملاحظة قديمة قبل البحث عن ملاحظات جديدة.

الملفات: `src/lib/appointment-slot.ts` · `visits.ts` · `src/lib/actions/appointments.ts` · `working-hours.ts` · `src/lib/data/appointments.ts`

حالة التقرير القديم: كل الملاحظات السبعة لا تزال قائمة

تم التأكد بقراءة الكود الحالي أن كل الثغرات التالية من التقرير السابق (`appointments-audit-report.html`) لم تُصلح بعد:

- ① شكل `workingHours` القديم (object) يُتجاهل ويرجع لساعات افتراضية 08:00-20:00 — `working-hours.ts:19`
- ② نموذج تعديل الفرع يسمح بالشكل القديم لساعات العمل عند الحفظ — `branch-form-dialog.tsx:87-89`
- ③ التقويم الأسبوعي يطابق الوقت كنص مطابقة تامة فيسقط أي موعد مش على نص/كامل الساعة — `appointments-calendar.tsx:131-136`
- ④ فحص التعارض والحجز عمليتان منفصلتان بلا atomic transaction
- ⑤ فحص ساعات العمل يتأكد من وقت البداية بس مش النهاية، فموعد 60 دقيقة الساعة قفل — race condition ممكن.
- ⑥ `canStaffAccessPatient` موجودة لكن غير مُستخدمة في الحجز/تسجيل الزيارة — موظف يعرف ID مريض غريب يقدر يحجزه في فرعه.
- ⑦ تسجيل زيارتين لنفس الموعد بضغط مزدوج لسه يبطلع خطأ Prisma خام بدل رسالة واضحة.

إلغاء/تغيير حالة موعد بعده Visit فعلية — بلا حارس

`updateAppointmentStatus` — `src/lib/actions/appointments.ts:178-210` يقبل أي حالة جديدة بغض النظر عن وجود `Visit` مسجلة مسبقًا. بعد ما "حضر" وتتسجل الزيارة (وربما فاتورة مرتبطة بيها)، لسه ممكن حد يقلب الموعد لـ "ملغي" أو "لم يحضر"، فتفضل الزيارة/الفاتورة "يتيمة" مقابل موعد ملغي بلا أي تسوية.

إلغاء مزدوج بيكرر الملاحظة وسجل التدقيق

`src/lib/actions/appointments.ts:212-243` — لا يوجد فحص أن الحالة الحالية ليست بالفعل CANCELLED قبل التنفيذ، فضغطة إلغاء مزدوجة (شبكة بطيئة + إعادة محاولة) بتضيف سطر "سبب الإلغاء" ثاني في notes وتنشئ AuditLog ثاني.

سبب الإلغاء/عدم الحضور مخزن كنص ملحق، مش عمود منفصل

`src/lib/actions/appointments.ts:188-190, 225` — السبب بيتلصق كنص داخل `notes`، وأي تعديل لاحق على الملاحظات عبر `updateAppointment` يستبدل النص كله ويفقد السبب المُسجل.

نقاط قوة مؤكدة

كشف التعارض الفعلي (`src/lib/data/appointments.ts:196-220`) يحسب تداخل الفترات الزمنية صح، وبيغطي كل فروع الطبيب مش فرع واحد بس — يمنع فعليًا حجز نفس الطبيب في فرعين في نفس الوقت. جدول `WeeklyAvailability` غير مستخدم حاليًا (مؤكد ومطابق لـ `TODO.md`) — مفيش تضارب بين ساعات الطبيب الشخصية وساعات الفرع لأن الأولى غير مفعلة أصلًا.

04 الفواتير + Workflow الموافقات

القاعدة الذهبية للنظام: الفاتورة مقفلة (isLocked) بعد الإصدار، وأي تعديل/حذف دفعة أو فتح فاتورة لازم يمر عبر ApprovalRequest يعتمد على SUPER_ADMIN. المراجعة أثبتت أن نقطة الالتفاف موجودة في تنفيذ الاعتماد نفسه، مش في الطلب.

الملفات: `src/app/(dashboard)/invoices/**` · `src/lib/actions/invoices.ts`

حرج

اعتماد "تعديل دفعة" يفتح الفاتورة كلها، مش الدفعة المطلوبة بس

— `updatePayment:268-299` + `src/lib/actions/invoices.ts:229-234` يعتمد SUPER_ADMIN لما
`EDIT_PAYMENT` ، الكود بيغير `isLocked: false` على الفاتورة فقط، من غير ما يسجل الدفعة المعتمدة تحديداً. بعدين
`updatePayment` بيقبل أي `paymentId` جاي من العميل وبيتحقق بس إن الفاتورة مش مقفولة — مفيش ربط بين
الموافقة والدفعة المحددة فيها.

سيناريو فعلي: الأدمن يوافق على تصحيح غلطة كتابية في دفعة أ. الفاتورة بتفتح بالكامل، وأي موظف مخوّل يقدر يعدّل دفعة ب
(كاش مثلاً) بدون ما يمر بأي موافقة أصلاً — ال workflow بيتلف بالكامل.

حرج

فتح الفاتورة (UNLOCK_INVOICE) بدون ضمان إعادة القفل

— `src/lib/actions/invoices.ts:211-212` الموافقة بنحط `isLocked: false` بشكل دائم؛ مفيش حاجة بتعيد
القفل غير استدعاء `updatePayment` لاحقاً (اللي بيقلها تلقائياً في نهايته). لو مفيش تعديل دفعة حصل فعلاً بعد الفتح، الفاتورة
بتفضل مفتوحة للأبد لأي موظف مخوّل.

متوسط

موافقات على كيانات محذوفة/متغيرة بتعتمد بصمت كأنها نجحت

— `src/lib/actions/invoices.ts:213-234` لو الدفعة اتمسحت بين وقت الطلب والاعتماد، الكود بيتخطى التنفيذ
بصمت (`if (payment) {...}`) لكن بيعلم الطلب APPROVED وبيسجل Audit Log كأن كل حاجة تمت. وفي حالة
`UNLOCK_INVOICE` على فاتورة اتمسحت، `tx.invoice.update` بيرمي خطأ Prisma خام (P2025) بدل فشل واضح.

متوسط

طلب الموافقة نفسه بلا تحقق من `entityId`

— `src/lib/actions/invoices.ts:173-198` `submitApprovalRequest` ما بيتحققش إن `entityId` فعلاً
دفعة/فاتورة موجودة، ولا إنها تخص فرع الموظف الطالب — بعكس `recordPayment` / `updatePayment` اللي بتستخدم
`canAccessBranch` . الواجهة بتقيّد الاختيار، لكن السيرفر نفسه بيثق في أي قيمة توصله.

متوسط

إمكانية الدفع الزائد (Overpayment) عبر تعديل دفعة

بيحدّ المبلغ الجديد بالمتبقي فعليًا، لكن `recordPayment` — `src/lib/actions/invoices.ts:268-299`
`updatePayment` (المستخدم بعد موافقة التعديل) بيقبل أي مبلغ بلا مقارنة بإجمالي الفاتورة، فالحالة بتتسبب PAID بدون أي تنبيه بوجود دفع زائد.

متوسط

تعارض محتمل في رقم تسلسل الفاتورة تحت الضغط المتزامن

`count+1` ثم يُنسّق `count` بيعمل `nextSequenceNumber` — `src/lib/actions/invoices.ts:25-29`
transaction، لكن تحت READ COMMITTED (الافتراضي في Postgres) عملينا إنشاء فاتورة متزامنتين ممكن يقرأوا نفس العدد قبل ال commit، فتتصادم مع ال unique constraint ويطلع خطأ Prisma غير معالج بدل إعادة محاولة.

بسيط

موديل الاسترداد (Refund) غير مفعّل نهائيًا

موديل `Refund` في ال Schema بيقرأ ويُعرض بس لو فيه صفوف موجودة (`[id]/page.tsx:210-224`) — لا يوجد أي Server Action أو واجهة لإنشاء استرداد فعليًا.

سليم

نقاط قوة مؤكدة

حساب subtotal/discount/total متسق والواجهة بتعيد حساب المتبقي من مجموع الدفعات الفعلية بدل الاعتماد على قيمة مخزّنة قابلة للانحراف. النطاق حسب الفرع (branch scoping) مُفعّل فعليًا على السيرفر في `getInvoices` / `createInvoice` / `updatePayment` / `recordPayment` — موظف فرع ب فعليًا لا يقدر يشوف أو يدفع فاتورة فرع أ. صلاحية اعتماد/رفض الطلبات محصورة فعليًا على السيرفر ب `isSuperAdmin`، مش بس إخفاء في الواجهة.

05 المخزون + التقارير

المخزون مبني ك CRUD أصناف فقط بدون نظام حركات فعلي، والتقارير المالية حقيقية لكن مبنية فوق بيانات مصروفات/مستحقات غير موجودة أصلًا.

الملفات: `src/lib/data/inventory.ts` · `reports.ts` · `src/lib/actions/inventory.ts`

حرج (فجوة ميزة)

لا يوجد نظام حركات مخزون على الإطلاق

— الموجود فعلاً هو `src/lib/actions/inventory.ts:1-94`
`deleteInventoryItem` / `updateInventoryItem` / `createInventoryItem` بس. لا يوجد `recordStockMovement` ولا أي أكشن إضافة/صرف/تحويل. موديل `StockMovement` بالكامل في ال Schema (إضافة، صرف، تحويل بين فروع) ميت — لا يوجد أي مكان في الكود بينشئ صف فيه.

سيناريو فعلي: الكمية الوحيدة اللي تتغير هي عن طريق الكتابة المباشرة فوق حقل `quantity` من نموذج التعديل — يعني مفيش سجل تدقيق لمين ضاف/صرف كام، ومفيش تحويل حقيقي بين الفروع، ومفيش حماية من إدخال رقم عشوائي يمحي

متوسط

لا يوجد تنبيه نقص مخزون على الداشبورد

– التنبيه الوحيد للمخزون هو تلوين الصف بالأحمر داخل صفحة المخزون نفسها (`src/lib/actions/dashboard.ts` ، والمقارنة "أقل من أو يساوي حد الطلب" صحيحة). لكن لو الأدمن مداخلش صفحة المخزون، معندوش أي إشارة على الداشبورد الرئيسي أن صنف وصل لحد إعادة الطلب – رغم إن الخطة الأصلية كانت تتوقع هذا التنبيه هناك.

بسيط

تاريخ الصلاحية مخزن لكن غير مُستغل فعليًا

– تلوين أحمر لو الصلاحية أقل من 30 يوم، محسوب في كل طلب من `Date.now()` . لا يوجد تمييز بين "منتهي فعلاً" و"قارب على الانتهاء"، ولا أي تنبيه خارج هذا الصف.

متوسط (يؤثر على دقة صافي الربح)

أرقام المصروفات والمستحقات في التقارير = صفر دائماً

– كود التجميع حقيقي وبيستعلم فعليًا من `prisma.expense` و `prisma.earningRecord` ، لكن لا يوجد أي ملف `expenses.ts` أو `earnings.ts` في `src/lib/actions/` ولا أي واجهة لإدخال مصروف أو احتساب مستحقات. النتيجة: `totalExpenses` و `totalEarnings` هيفضلوا صفر دائماً في نظام حقيقي، وبالتالي "صافي الربح" المعروض مش دقيق.

متوسط

عدم اتساق التوقيت الزمني في حدود نطاق التقارير

– بداية النطاق (`sp.from`) بتتحلل كـ UTC منتصف الليل بينما النهاية الافتراضية بتوقيت محلي، ودالة `endOfDay` في `reports.ts:5-9` بتوقيت محلي. في توقيت مصر (+2/+3)، ده بيزيح حد البداية 2-3 ساعات عن منتصف الليل المحلي الفعلي، وبيختلف السلوك بين نطاق افتراضي ونطاق مُدخّل يدويًا.

سليم

نقاط قوة مؤكدة

صلاحية SUPER_ADMIN على المخزون مُفعّلة فعليًا على السيرفر (مش بس إخفاء واجهة) في كل من الصفحة وكل Server Action. أرقام الإيرادات والمواعيد والمرضى في التقارير حقيقية 100% ومحسوبة من بيانات فعلية، ومُصدّرة PDF بنفس الأرقام الحقيقية بدون قيم وهمية.

06 غير مبني حتى الآن

موديولات موجودة في الخطة وال Schema لكن بلا أي كود تنفيذ – لمعرفة حدود ما تم إنجازه بدقة.

فقط Schema

المصروفات (Expense)

موديل `Expense` موجود في `schema.prisma:446-459` ، بدون أي Server Action أو صفحة لإدخال أو عرض مصروف.

فقط Schema

مستحقات الأطباء (EarningRule / EarningRecord)

لا يوجد أي كود لاحتساب أو عرض مستحقات الأطباء/الموظفين رغم وجود الموديلين بالكامل في ال Schema بما في ذلك آلية تاريخ التسعير (`previousRuleId`).

فقط Schema

جدولة الطبيب الأسبوعية (WeeklyAvailability)

الموديل موجود لكن غير مستخدم في أي مكان — الحجز حاليًا يعتمد فقط على ساعات عمل الفرع، مش جدول الطبيب الشخصي.

غير موجودة

صفحة الإعدادات (Settings)

لا يوجد `app/(dashboard)/settings/` — إدارة المستخدمين والفروع تمت بالفعل كصفحات مستقلة (`/doctors` , `/assistants` , `/branches`) بدل تجميعها هناك، لكن "الخدمات والأسعار" و "الإشعارات" من الخطة الأصلية غير موجودين إطلاقًا.

★ ترتيب الأولوية للإصلاح

لو هيتاخذ إجراء واحد بس قبل أي حاجة ثانية، يبدأ من هنا — الترتيب حسب الأثر المباشر على أمان بيانات المرضى والمال.

- 1 إضافة فحص `getCurrentUser` + `canStaffAccessPatient` / `canAccessBranch` لكل من `createPatient` , `updatePatient` , `upsertMedicalHistory` , `updateToothRecord` — هذه أخطر 4 ثغرات في المشروع كله.

patients.ts · medical-history.ts · dental-chart.ts

- 2 ربط اعتماد `UNLOCK_INVOICE` / `EDIT_PAYMENT` بالدفعة المحددة نفسها بدل فتح الفاتورة كلها، وضمان إعادة القفل تلقائيًا بعد أي عملية.

invoices.ts:203-234

- 3 إعادة التحقق من `isActive` والدور على كل طلب (أو تقصير مدة الجلسة) بدل الاعتماد على JWT ثابت طول شهر.
auth.ts

- 4 منع تغيير حالة موعد له Visit مسجلة بالفعل إلى "ملغي"/"لم يحضر" بدون تسوية صريحة للزيارة/الفاتورة المرتبطة.

5 بناء نظام حركات المخزون الفعلي (إضافة/صرف/تحويل) بدل الكتابة المباشرة فوق الكمية، وربط تنبيه نقص المخزون بالداشبورد.

inventory.ts

تم إعداد هذا التقرير بمراجعة الكود مباشرة (قراءة فقط، بدون أي تعديل) – يغطي كل الموديولات المبنية فعليًا في المستودع حتى تاريخ التقرير. الملاحظات الخاصة بموديول المواعيد مبنية جزئيًا على مقارنة مع تقرير [appointments-audit-report.html](#) السابق.